

Biometric Fuzzy Extractor & Post-Quantum Cryptography

The Technical Guide to Zero-Knowledge Enrollment and Authentication Workflows

1. Introduction & The Core Concepts

Traditional security keys (like passwords or keys stored on disk) require an **exact matching** input. If you type a single character wrong, access is denied. However, human biometrics (such as fingerprints) never yield identical scans. Skin moisture, pressure, dirt, cuts, and finger rotation cause slight variations every time the print is scanned.

A **Fuzzy Extractor** solves this problem. It acts as a biometric translator, using advanced error-correction math to reconstruct the exact same cryptographic key from a slightly noisy fingerprint scan. Furthermore, it operates under the **Zero-Knowledge** principle: the database never stores raw fingerprint templates or features. Instead, it stores mathematical puzzles (Helper Data) and a verification stamp (Hash). If the database is compromised, the attacker cannot reverse-engineer or reconstruct the user's physical fingerprint.

Key Security Note: The system combines a Fuzzy Extractor with NIST FIPS 203 standardized **ML-KEM-768** Post-Quantum Cryptography to establish keys that remain secure even against future quantum computer attacks.

2. Phase 1: Enrollment (User Registration)

Enrollment is the process of generating a secure locked box and registering it on the server. The steps are:

- **Step 1: Fingerprint Capture**

The user places their finger on the sensor. The system captures the raw image.

- **Step 2: Feature Extraction**

The image is processed using Local Binary Patterns (LBP) and Gabor filters to extract a unique 71-byte list of numbers representing the print's textures and directions (features X). Features are quantized to uint8 integers to smooth out minor floating-point differences.

- **Step 3: Random Key Generation**

The system generates a random secret seed K (representing the core cryptographic key).

- **Step 4: Error-Correction Codeword**

The system expands the secret key K into a 120-byte Reed-Solomon Codeword (C). This codeword contains redundant mathematical parity symbols allowing it to survive up to 30 corrupted/changed bytes.

- **Step 5: Puzzle Generation (Helper Data P)**

The system XORs the codeword C with the fingerprint features X to form the Helper Data: $P = C \text{ XOR } X$. On its own, P is indistinguishable from random noise and contains zero information about the fingerprint.

- **Step 6: Verification Hash**

The key K is run through SHA-256 to create a verification stamp $v = H(K)$, allowing future confirmation without storing the key itself.

- **Step 7: Post-Quantum Keypair Generation**

An ML-KEM-768 public/private keypair is generated to secure all future session agreements.

- **Step 8: Save & Purge**

The database saves the Helper Data P , the verification hash v , and the PQC keys. The raw fingerprint image and extracted features X are **deleted permanently from memory**.

3. Phase 2: Authentication (Login & Key Agreement)

Authentication is the process of using the user's finger to unlock the stored puzzle and establish a secure quantum-safe line:

- **Step 1: Fresh Capture**

The user scans their finger again. The captured image yields a new feature list X' which differs slightly from the original X .

- **Step 2: 10-Orientation Search**

Because the finger may be placed at different angles, the system generates 10 rotated variations of X' spaced by 36 degrees.

- **Step 3: Codeword Recovery Attempt**

For each angle, the system XORs the candidate features with the stored helper data: $C' = P \text{ XOR } X'$.

- **Step 4: Error Correction**

The system feeds C' into the Reed-Solomon decoder. If the orientation is correct, the decoder corrects the errors (up to 30) and recovers the exact original key K .

- **Step 5: Cryptographic Hash Check**

The recovered key is hashed and matched against the stored stamp v . If they match, identity is mathematically proven.

- **Step 6: Post-Quantum Encapsulation**

The server uses the user's stored ML-KEM-768 public key to encapsulate a shared secret, generating a ciphertext block c and session seed ss .

- **Step 7: Post-Quantum Decapsulation**

The client's secure private key decapsulates c to recover ss' .

- **Step 8: Final Session Key**

Both sides combine the biometric key and the post-quantum secret: $K_{session} = SHA256(R || ss)$, guaranteeing confidentiality secure against classical and quantum attackers.